UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2012/2013

APRIL EXAMINATION

UECS3253 WIRELESS APPLICATION DEVELOPMENT

WEDNESDAY, 24 APRIL 2013 TIME : 2.00 PM – 4.00 PM (2 HOURS)

BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING

Instruction to Candidates :

This Examination Paper consists of **FIVE (5)** questions and is divided into **TWO (2)** sections.

Section A: (Q1 & Q2) [Total: 50 marks]

Q1 and Q2 are compulsory.

Section B: (Q3 to Q5) [Total: 50 marks]

Answer **ANY TWO (2)** out of the **THREE (3)** questions given in this section. Each question carries 25 marks.

If more than two questions are answered in Section B, **only the first TWO (2)** answers will be marked and taken into account.

An Appendix is provided at the end of this question paper which provides a listing of classes and methods that you can utilize in your answers.

UECS3253 WIRELESS APPLICATION DEVELOPMENT

SECTION A: Answer ALL questions.

Q1. (a) One way for persisting data in Android is through the use of a native database such as the SQLite database system. To create and access a SQLite database, the SQLiteOpenHelper class is usually extended and the existing methods in that class are overridden with new implementations. A table with the name of HRFiles is to be created as shown below, with EmployeeNum being the designated primary key.

Column name	Column Type
EmployeeNum	Integer
Name	Text
Age	Integer

(i) Figure 1 (Q1) shows part of a private class DatabaseHelper that extends SQLiteOpenHelper. Identify the method that must be overridden in DatabaseHelper to create the table HRFiles and write the implementation for it. If any static variables are used in this method, they must be first defined correctly in accordance to the table shown above. (4 marks)

(ii) Provide an implementation and appropriate argument list for a method that will insert a new record into the table. (5 marks)

(iii) An existing record in the table with a Name field of Peter has an incorrect value of 65 in the Age field. This needs to be amended to the correct value of 55. Provide the implementation and argument list for two (2) additional methods in DatabaseHelper called getContact(...) and updateContact(...) that can be used to help perform this amendment. (10 marks)

(iv) Write an appropriate sequence of statements to invoke these two methods in order to perform the amendment described in (iii). You may assume that there is only one record in the table that has the Name field with value Peter. (4 marks)

(b) Explain how you can pre-create your own SQLite database using an existing database browser tool, and then have this database installed with your Android application to a mobile device. No code writing is necessary; a brief description of the procedure will suffice. (2 marks)

[Total : 25 marks]

```
public class DBAdapter {

        final Context context;
        DatabaseHelper DBHelper;
        SQLiteDatabase db;
        public DBAdapter(Context ctx)
        {
                this.context = ctx;
                DBHelper = new DatabaseHelper(context);
        }
        private static class DatabaseHelper extends SQLiteOpenHelper
        {
                DatabaseHelper(Context context)
                {
                        super(context, "UTARdatabase", null, "1");
                }
                ........................................
        }
```

Figure 1 (Q1)

Q2. (a) A simple Android application is designed in such a way that when the button "Display second activity" in the first activity (UsingIntentActivity) is clicked, another activity (SecondActivity) will be loaded with the message "This is the Second Activity" as shown in Figure 2 (Q2). The AndroidManifest.xml of the application is shown in Figure 3 (Q2). In UsingIntentActivity.java, the onClick() method is defined as below:

```
public void onClick(View view) {
        startActivity(new Intent("net.learn2develop.SecondActivity"));
}
```

(i) What will happen when UsingIntentActivity is loaded and the button "Display second activity" is clicked ? (3 marks)

(ii) What is the purpose of the Intent class ? (3 marks)

(iii) What is the purpose of the intent-filter element ? (3 marks)

Q2. (Continued)

(b) A developer must configure an application correctly before it can be published on a public application store such as Google Play or Android Market. Describe in detail the purpose of each of the following attributes below in this configuration process.

- android:versionCode
- android:versionName
- android:label
- android:minSdkVersion

(8 marks)

(c) Produce the complete application setting file: AndroidManifest.xml for the application project "ABC Project" based on the information provided in Figure 4 (Q2) and Figure 5 (Q2). Your answer should include the following tags and attributes shown below:

- <manifest>
- <uses-sdk>
- <application>
- <activity>
- <intent-filter>
- <action>
- <category>
- package
- android:versionCode
- android:versionName
- android:minSdkVersion
- android:targetSdkVersion
- android:icon
- android:label
- android:name

(8 marks)
[Total : 25 marks]

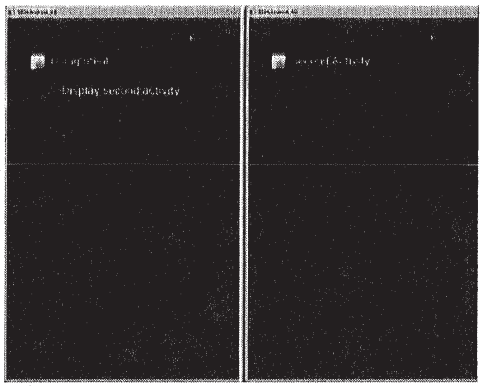


Figure 2 (Q2)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
        package="net.learn2develop.UsingIntent"
        android:versionCode="1"
        android:versionName="1.0" >

        <uses-sdk  android:minSdkVersion="14" />

        <application
                android:icon="@drawable/ic_launcher"
                android:label="@string/app_name" >
                <activity
                        android:label="@string/app_name"
                        android:name=".UsingIntentActivity" >
                        <intent-filter >
                            <action android:name="android.intent.action.MAIN" />
                            <category
                            android:name="android.intent.category.LAUNCHER" />
                        </intent-filter>
                </activity>
                <activity
                        android:label="ActivityTwoPointOne"
                        android:name=". ActivityTwoPointOne" >
                        <intent-filter >
                            <action android:name="net.learn2develop.SecondActivity"
                            />
                            <category
                            android:name="android.intent.category.DEFAULT" />
                        </intent-filter>
                </activity>
                <activity
                        android:label=" ActivityTwoPointTwo"
                        android:name=". ActivityTwoPointTwo" >
                        <intent-filter >
                            <action android:name="net.learn2develop.SecondActivity"
                            />
                            <category
                            android:name="android.intent.category.DEFAULT" />
                        </intent-filter>
                </activity>
        </application>
</manifest>
```

Figure 3 (Q2)

- Package name: net.learn2develop.ABCProject
- Application icon: @drawable/ic_launcher
- Targeted Android version: 4.0
- Minimum supported Android version: 3.0
- The package contains only one activity named "Main"
- The package is the second version after some minor updates on version 1

```
<resources>

  <string name="app_name">ABC Project</string>
  <string name="menu_settings">Settings</string>

</resources>
```

Figure 4 (Q2)

Platform Version	API Level	VERSION_CODE
Android 4.2	17	JELLY_BEAN_MR1
Android 4.1, 4.1.1	16	JELLY_BEAN
Android 4.0.3, 4.0.4	15	ICE_CREAM_SANDWICH_MR1
Android 4.0, 4.0.1, 4.0.2	14	ICE_CREAM_SANDWICH
Android 3.2	13	HONEYCOMB_MR2
Android 3.1.x	12	HONEYCOMB_MR1
Android 3.0.x	11	HONEYCOMB
Android 2.3.4 Android 2.3.3	10	GINGERBREAD_MR1
Android 2.3.2 Android 2.3.1 Android 2.3	9	GINGERBREAD
Android 2.2.x	8	FROYO
Android 2.1.x	7	ECLAIR_MR1
Android 2.0.1	6	ECLAIR_0_1
Android 2.0	5	ECLAIR
Android 1.6	4	DONUT
Android 1.5	3	CUPCAKE
Android 1.1	2	BASE_1_1

Figure 5 (Q2)

SECTION B: Answer ONLY 2 questions

Q3. (a) A client-server application is developed using the sockets model for network programming. The client application is to be deployed on an Android mobile device, while the server application will run on a standard desktop operating system environment. The main activity file for the client application is called SocketsActivity. It uses a private CreateCommThreadTask class to asynchronously create a socket connection with the server. The CreateCommThreadTask in turn uses another class called CommsThread to process this socket connection in a threaded fashion. SocketsActivity also includes a static handler class that updates its own UI with an incoming message that is received from the server. The class variables for SocketsActivity are shown in Figure 6 (Q3). The layout file main.xml for SocketsActivity is shown in Figure 7 (Q3).

(i) Write down the statement that needs to be added to the AndroidManifest.xml file of the client application in order for it to operate properly. (2 marks)

(ii) Provide the code snippet for the static handler class within SocketsActivity. This class should include a single method with a signature as follows: public void handleMessage(Message msg). You can assume that the TextView element in the UI will be used in this method for displaying the message passed to the static handler class. (5 marks)

(b) CreateCommThreadTask will create a new CommsThread object by passing it a socket object that points to the IP address of the server application. The CommsThread class must create the appropriate objects to handle input/output for this socket, as well as update the static handler that you created in part Q3 (a) (ii) every time an incoming message is received on the socket.

(i) Write two (2) snippets of code showing the constructor of CommThread as well as its run() method. (6 marks)

(ii) Write the code snippet for the doInBackground(Void ....params) method of CreateCommThreadTask that will create and start the CommsThread object. You may assume that the server application is listening to port 888 at IP address 192.168.1.1. (4 marks)

Q3. (Continued)

(c) SocketsActivity should also be able to send a message asynchronously to the server application. This will happen when the user enters data into a EditText field and presses the Send Message button on the main UI screen. The actual message send operation will be implemented in the CommsThread class.

(i) Write the code snippet for a method to be implemented in CommThread that accepts an array of bytes and sends this to the server application. (2 marks)

(ii) Write the appropriate methods and statements necessary in SocketsActivity to extract the user's data entry from the EditText field when the Send Message button is pressed, and pass this data asynchronously to the method in CommThread that you have just implemented in (c) (i). (6 marks)

[Total : 25 marks]

```
import java.io.IOException;
import java.net.InetAddress;
import java.net.Socket;
import java.net.UnknownHostException;
import android.app.Activity;
import android.os.AsyncTask;

import android.os.Bundle;
import android.os.Handler;
import android.os.Message;
import android.view.View;
import android.widget.EditText;
import android.widget.TextView;


public class SocketsActivity extends Activity {

        InetAddress serverAddress;
        Socket socket;
        static TextView txtMessagesReceived;
        EditText txtMessage;
        CommsThread commsThread;
        .......
        .......
```

Figure 6 (Q3)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:orientation="vertical" >

<EditText
        android:id="@+id/txtMessage"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content" />

<Button
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Send Message"
        android:onClick="onClickSend"/>

<TextView
        android:id="@+id/txtMessagesReceived"
        android:layout_width="fill_parent"
        android:layout_height="200dp"
        android:scrollbars = "vertical" />

</LinearLayout>
```

Figure 7 (Q3)

Q4. (a) An Android application is initially created for the purpose of sending SMSes. The main activity for this application is SMSActivity. The initial layout main.xml for the application is shown in Figure 8 (Q4), while the initial AndroidManifest.xml is shown in Figure 9 (Q4).

(i) Write a snippet of code within the SMSActivity class to send a SMS with the text "I am going to pass this exam !" to the number 0123456789 when the Send SMS button is pressed on the main UI screen. (4 marks)

(ii) Modify your snippet of code in Q4 (a) (i) so that the SMS is not sent by the SMSActivity class, but by an existing native SMS application previously installed on the mobile device. (5 marks)

Q4. (Continued)

(b) The application from Q4 (a) is to be extended to be able to receive incoming SMSes by incorporating a receiver class by the name of SMSReceiver. Incoming SMSes should be processed by both the application as well as any existing native SMS application previously installed on the mobile device.

(i) Write the additional statements that need to be included in the original AndroidManifest.xml as shown in Figure 9 (Q5). (4 marks)

(ii) Write a complete definition for the onReceive() method in the SMSReceiver class. This method will receive the SMS data via an Intent object passed in as an argument, and extract it using the Bundle and SmsMessage classes within the Android API. The extracted SMS text should be flashed as a message on the screen using the Toast widget. (8 marks)

(iii) The application can be further modified to ensure that SMSReceiver is the first activity to process any incoming SMS, and that no other native SMS applications are invoked additionally. Describe in detail using code snippets any further modifications you need to make to the application to achieve this purpose. (4 marks)

[Total : 25 marks]

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"
        android:orientation="vertical" >

<Button
        android:id="@+id/btnSendSMS"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="Send SMS"
        android:onClick="onClick" />
</LinearLayout>
```

Figure 8 (Q4)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
package="net.learn2develop.SMS"android:versionCode="1"
android:versionName="1.0" >
<uses-sdk android:minSdkVersion="10" />

<uses-permission android:name="android.permission.SEND_SMS"/>

<application
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name" >
        <activity
                android:label="@string/app_name"
                android:name=".SMSActivity" >
                <intent-filter >
                        <action android:name="android.intent.action.MAIN" />
                        <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
        </activity>


</application>
</manifest>
```

Figure 9 (Q4)

Q5. (a) Elaborate on the differences between a mobile native application and a mobile Web application from the following perspectives:

- ability to access device's native features
- method of delivery or deployment
- software updates
- monetization or revenue generation from the app
- advantages

(5 marks)

Q5. (Continued)

(b) A user goes through the sequence of steps described below while interacting with the built-in Dialer application on his Android phone. Draw an activity lifecycle flow chart showing the various methods that can be called, as well as the different states that the Dialer application can go through. Identify the state that the Dialer application is in for each step (i) to (vii) below. Number the transitions between states accordingly in your flow chart. The screenshots of his phone display for these steps are shown in Figure 10 (Q5).

(i) The user touches on the Dialer icon to bring up the Dialer.

(ii) The user dials a phone number on the screen keypad and touches the "Call" icon.

(iii) The user ends the call.

(iv) The user presses the "Back" button.

(v) The user goes on to touch on the "Chrome" icon to bring up the browser.

(vi) The user goes to Settings > Applications > Manage applications > Running.

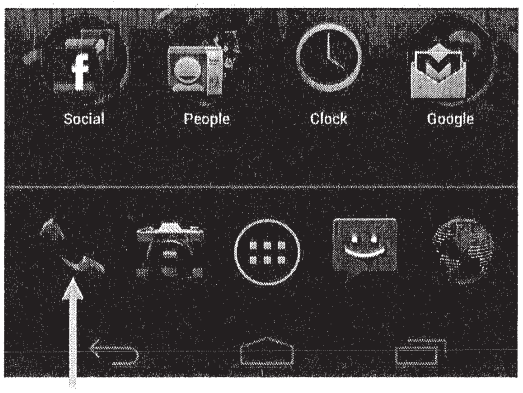
(vii) The user forces the Dialer to stop.

(10 marks)

(c) Create an activity class named "MainActivity" which extends the base class "Activity" in a package named "com.example.Main". Write the code for a callback method which binds this activity to the main.xml layout file when the activity is loaded. Include all necessary import statements in your class declaration. (5 marks)

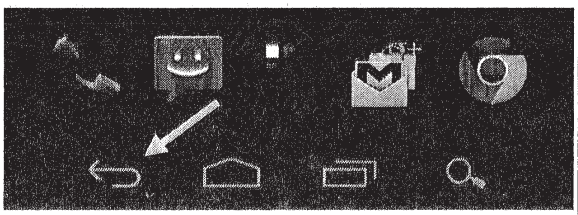
(d) The layout of "MainActivity" is shown in the main.xml file in Figure 11 (Q5). Sketch the user interface corresponding to this layout file when the activity is loaded. (5 marks)

[Total : 25 marks]

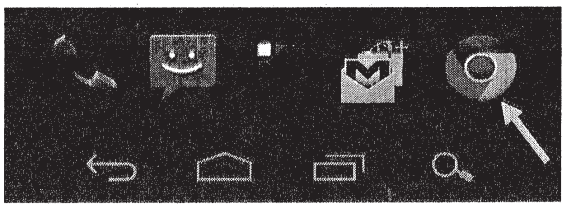


Q5 b) i)

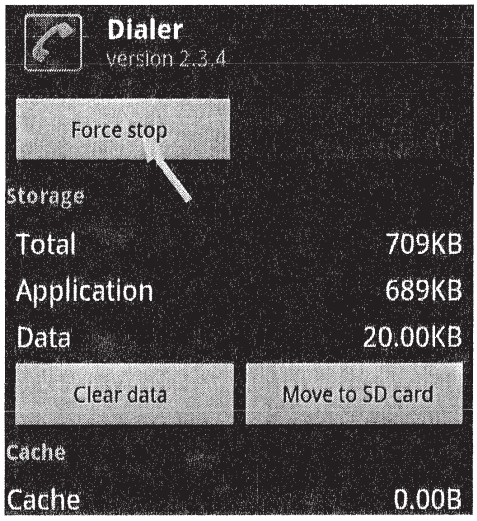
Q5 b) ii)



Q5 b) iv)



Q5 b) v)



Q5 b) vii)

Figure 10 (Q5)

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:id="@+id/root"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  android:orientation="vertical" >
      <LinearLayout
            android:orientation="horizontal"
            android:layout_width="fill_parent"
            android:layout_height="wrap_content">
            <EditText
              android:id="@+id/lefttext"
              android:layout_width="fill_parent"
              android:layout_height="fill_parent"
              android:layout_weight="1"
              android:inputType="text"
              android:text="Left" />
            <EditText
              android:id="@+id/righttext"
              android:layout_width="fill_parent"
              android:layout_height="fill_parent"
              android:layout_weight="1"
              android:inputType="text"
              android:text="Right" />
      </LinearLayout>
            <LinearLayout
            android:orientation="horizontal"
            android:layout_width="fill_parent"
            android:layout_height="wrap_content">
            <Button
              android:id="@+id/leftbutton"
              android:layout_width="fill_parent"
              android:layout_height="fill_parent"
              android:layout_weight="1"
              android:text="Left" />
            <Button
              android:id="@+id/rightbutton"
              android:layout_width="fill_parent"
              android:layout_height="fill_parent"
              android:layout_weight="1"
              android:text="Right" />
      </LinearLayout>
</LinearLayout>
```

Figure 11 (Q5)

APPENDIX

The SQLiteOpenHelper class has the following useful methods:

- public void onCreate(SQLiteDatabase db) – creates a new SQLiteDatabase object
- public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) – upgrades the db object from oldVersion to newVersion

The SQLiteDatabase class has the following useful methods:

- execSQL(String s) – executes a SQL statement given by s
- insert(String table, String nullColumnHack, ContentValues values) – inserts into a given table, the value contained within the values object and returns the row ID of the newly inserted row.
- update (String table, ContentValues values, String whereClause, String[] whereArgs) – updates rows in table with values, where the rows are selected using the whereClause and returns the number of rows successfully updated in this manner.
- public Cursor query (boolean distinct, String table, String[] columns, String selection, String[] selectionArgs, String groupBy, String having, String orderBy, String limit) – query with a Cursor returned pointing to the results
  - distinct - true if you want each row to be unique, false otherwise.
  - table - The table name to compile the query against.
  - Columns - A list of which columns to return..
  - Selection - A filter declaring which rows to return, formatted as an SQL WHERE clause.
  - selectionArgs - You may include ?s in selection, which will be replaced by the values from selectionArgs, in order that they appear in the selection..
  - groupBy - A filter declaring how to group rows, formatted as an SQL GROUP BY clause.
  - having - A filter declare which row groups to include in the cursor, if row grouping is being used, formatted as an SQL HAVING clause.
  - orderBy - How to order the rows, formatted as an SQL ORDER BY clause (excluding the ORDER BY itself).
  - limit - Limits the number of rows returned by the query, formatted as LIMIT clause. Passing null denotes no LIMIT clause.

The ContentValues class has the following useful methods:

- put(String key, XXXXX value) - Adds a value matching to a given key, where the value can be of type Byte, Integer, Float, Short, byte[], String, Double, etc….

The Cursor class has the following useful methods:

- boolean moveToFirst() - Move the cursor to the first row.
- boolean moveToLast() - Move the cursor to the last row.
- boolean moveToNext() - Move the cursor to the next row.
- boolean moveToPrevious() - Move the cursor to the previous row.
- boolean moveToPosition(int position) - Move the cursor to an absolute position.
- long getLong(int columnIndex) - Returns the value of the requested column as a long.

- int getPosition() - Returns the current position of the cursor in the row set.
- String getString(int columnIndex) - Returns the value of the requested column as a String.

The String class has the following useful methods:

- String(byte[] array) – creates a String based on the contents of a byte array
- String substring(int beginIndex, int endIndex) - Returns a new string that is a substring of this string. The substring begins at the specified beginIndex and extends to the character at index endIndex - 1. Thus the length of the substring is endIndex-beginIndex.
- int toInteger() – converts the String to an equivalent Integer
- byte[] getBytes() – returns the String as an array of bytes

The Socket class

- Socket(InetAddress address, int port) – constructor that creates a stream socket and connects it to the specified port number at the specified IP address.
- getInputStream() – returns an InputStream object linked to that socket
- getOutputStream()– returns an OutputStream object linked to that socket

The InputStream class has the following useful methods:

- int read(byte[] b) throws IOException - Reads some number of bytes from the input stream and stores them into the buffer array b. The number of bytes actually read is returned as an integer.

The Output stream class has the following useful methods:

- write(byte[] b) throws IOException – Writes to the output stream the contents of the buffer array b.

The InputHandler class has the following useful methods:

- obtainMessage (int what, int arg1, int arg2, Object obj) - Returns a new Message, which has the following parameters set within that Message:
  - what Value to assign to the returned Message.what field.
  - arg1 Value to assign to the returned Message.arg1 field.
  - arg2 Value to assign to the returned Message.arg2 field.
  - obj Value to assign to the returned Message.obj field.

The class Message has the particular fields and useful methods:

- arg1, arg2: two integer variables to hold any particular value
- obj: an Object field to hold the contents of that message
- sendToTarget() - Sends this Message to the Handler associated with the Message

The InetAddress has the following useful methods:

- getByName(String host) – Returns the IP address of a host, given the host's name.

The Thread has the following useful methods:

- start() - Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread.

The AsyncTask has the following useful methods:

- execute(Params .... Params) – Starts the task with the passed params, which results in the doInBackGround(Params .... Params) method to be called.

The Activity class has the following useful methods:

- findViewById (int id) - Finds a view that was identified by the id attribute from the XML that was processed in onCreate(Bundle). Example of an id attribute is R.id.dog, R.id.cat, etc. Examples of views are TextView, EditText, Button, etc.
- startActivity(Intent i) – Used to start a new activity associated with the Intent i

The EditText and TextView class has the following useful methods:

- getText() - Return the text associated with that particular View class as a CharSequence object, which itself has a toString() method that returns a String with the same characters in the same order as the object
- setText(String s) – Puts s into the TextView

The SMSManager class has the following useful methods:

- getDefault() – returns the default instance of the SmsManager
- sendTextMessage (String destinationAddress, String scAddress, String text, PendingIntent sentIntent, PendingIntent deliveryIntent) - Send a text based SMS with the following parameters:
  - destinationAddress - the address to send the message to
  - scAddress - is the service center address or null to use the current default SMSC
  - text the body of the message to send
  - sentIntent - if not NULL this PendingIntent is broadcast when the message is successfully sent, or failed. The result code will be Activity.RESULT_OK for success,
  - deliveryIntent - if not NULL this PendingIntent is broadcast when the message is delivered to the recipient. The raw pdu of the status report is in the extended data ("pdu").

The Intent class has the following useful methods and static constants:

- ACTION_MAIN
- ACTION_VIEW
- ACTION_DIAL
- ACTION_EDIT
- ACTION_ANSWER
- putExtra (String name, xxx value) – puts a value of type xxxx to be associated with name in the Intent, where xxx can be int, String, char, Bundle, int[], float, byte[], etc
- setType (String type) – sets an explicit MIME type to be associated with the Intent.
- getExtras() – returns a Bundle object containing data previously put into the Intent object

The Bundle class has the following useful methods:

- get(String key) - Returns the entry with the given key as an Object.

The SmsMessage has the following useful methods:

- createFromPdu (byte[] pdu) – Creates a SmsMessage object from a raw PDU.
- getOriginatingAddress () - Returns the originating address (sender) of this SMS message in String form or null if unavailable
- getMessageBody () - Returns the message body as a String, if it exists and is text based.

The Toast class can be used to display messages in this form:

- Toast.makeText(context, String str, Toast.LENGTH_LONG).show();

where str is the message to be displayed.